End-to-End Implementation of Site Reliability Engineering Practices in a Multi-Orchestrated Microservices Infrastructure Using Docker Swarm, Kubernetes, Terraform, and Ansible

Comprehensive SRE End-Term Project Report

Distributed Microservices System Demonstration

Prepared as a final academic and technical submission

This report documents the complete SRE lifecycle of the NexShop microservices platform, including architecture design, dual orchestration, observability, incident response, automation, and capacity planning.

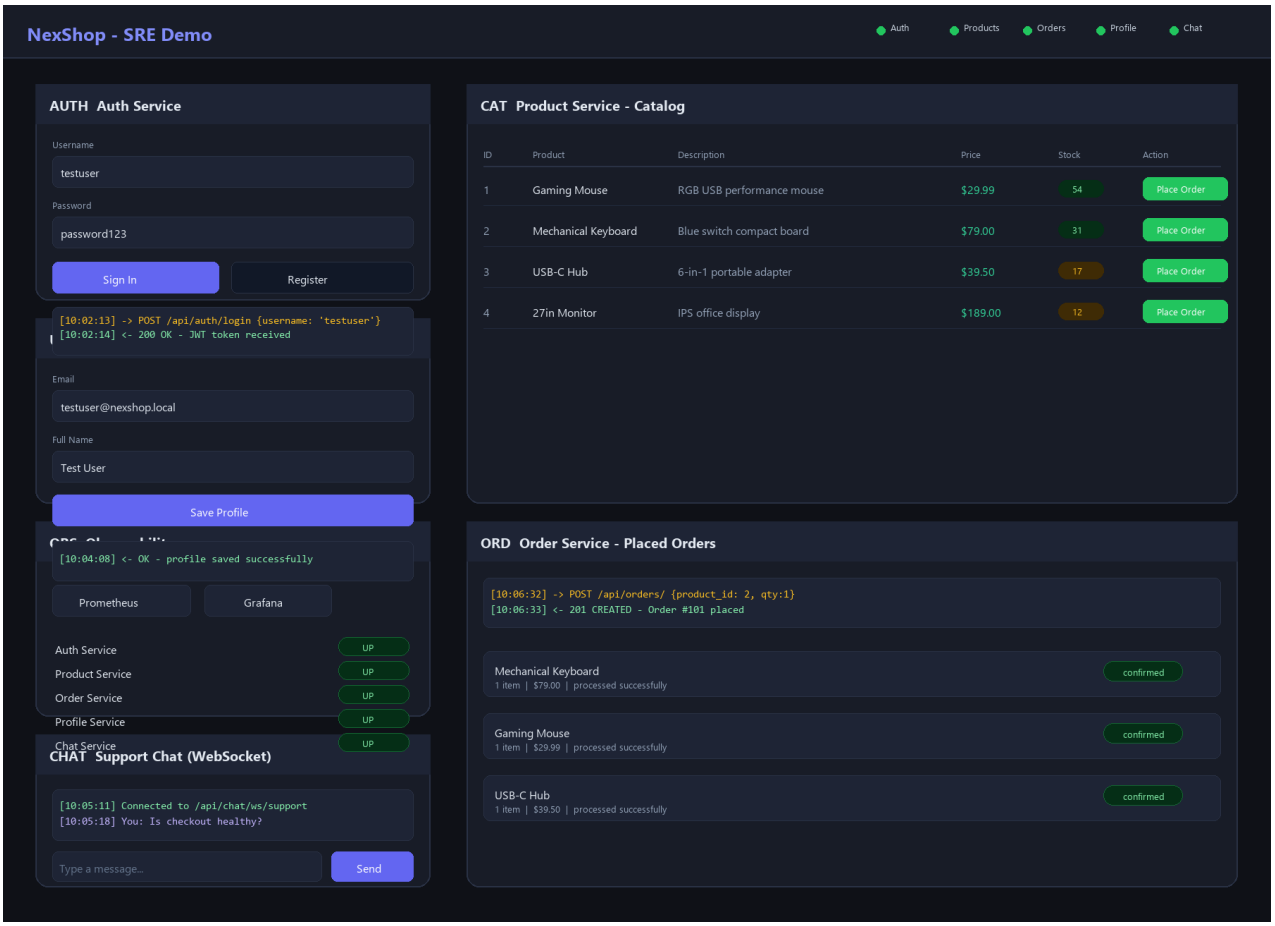

Figure 1. NexShop project homepage used to demonstrate service interaction, live health status, and observability entry points.

1. Abstract

In this end-term project, I implemented a complete Site Reliability Engineering workflow around a distributed microservices application named NexShop. The system was designed as a realistic web-based platform composed of more than six independently deployable services, including authentication, product catalog management, order processing, payment simulation, notification and chat support, and user profile management. To demonstrate orchestration maturity rather than a single deployment style, I deployed the same application using both Docker Swarm and Kubernetes and documented the operational differences between the two environments.

Infrastructure reproducibility was handled through Terraform, while Ansible automated operating system preparation, Docker installation, Swarm bootstrap, K3s installation, image import, Kubernetes deployment, and observability setup. Prometheus and Grafana were integrated to collect service-level and infrastructure-level metrics and to evaluate reliability using clearly defined SLIs and SLOs. I also simulated a production-style incident in the Order Service by introducing a database configuration problem, then used monitoring and logs to diagnose the issue, recover the service, and write a structured postmortem. Overall, this project demonstrates the full SRE lifecycle: design, provisioning, deployment, monitoring, incident response, automation, and capacity planning.

2. Objectives

The primary goal of this project was to show how SRE practices can be applied across the full lifecycle of a microservices system rather than being treated as an afterthought after deployment. I specifically focused on turning the application into an observable, reproducible, resilient, and scalable platform.

1. Design and deploy a distributed microservices architecture containing more than six services.
2. Implement multi-orchestration using both Docker Swarm and Kubernetes.
3. Provision infrastructure using Terraform as declarative Infrastructure as Code.
4. Automate configuration and deployment using Ansible playbooks.
5. Define practical SLIs and SLOs aligned with user-visible system behavior.
6. Implement monitoring and alerting using Prometheus and Grafana.
7. Simulate, detect, diagnose, and recover from a production-style incident.
8. Conduct a postmortem analysis focused on reliability learning rather than blame.
9. Apply automation and capacity planning strategies to improve scalability and maintainability.

3. System Overview

The system was built as a scalable web application that exposes a single Nginx-based frontend while routing traffic internally to multiple backend services. I intentionally designed the platform so that each service could be reasoned about independently while still participating in an end-to-end customer workflow. The frontend offers a compact operational demo surface where authentication, product listing, order placement, profile updates, WebSocket chat, and health visualization can all be exercised from one page.

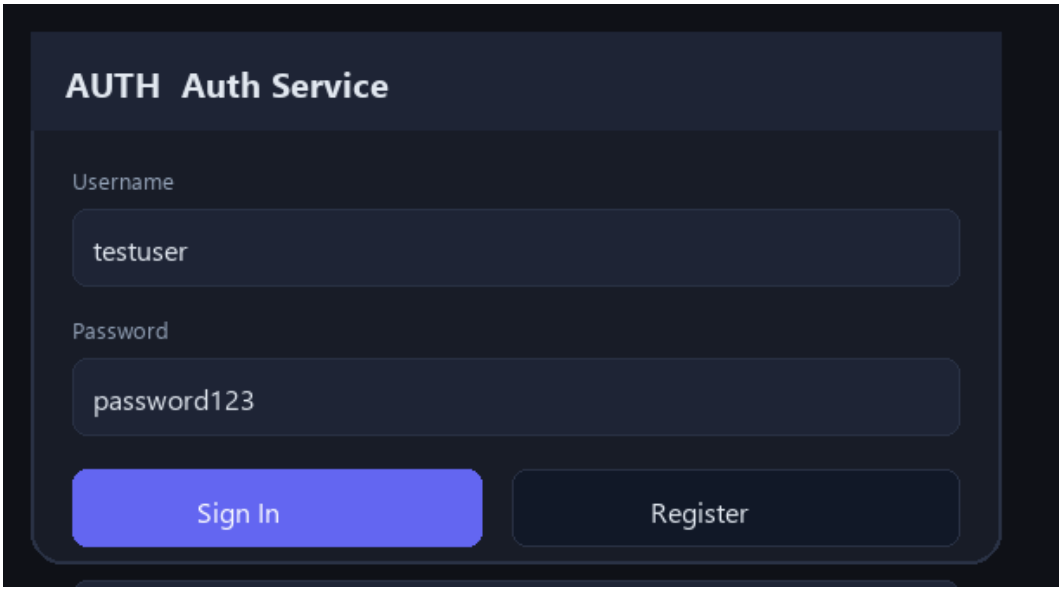

Figure 2. Authentication service interface used to demonstrate login, registration, and token issuance through the frontend gateway.

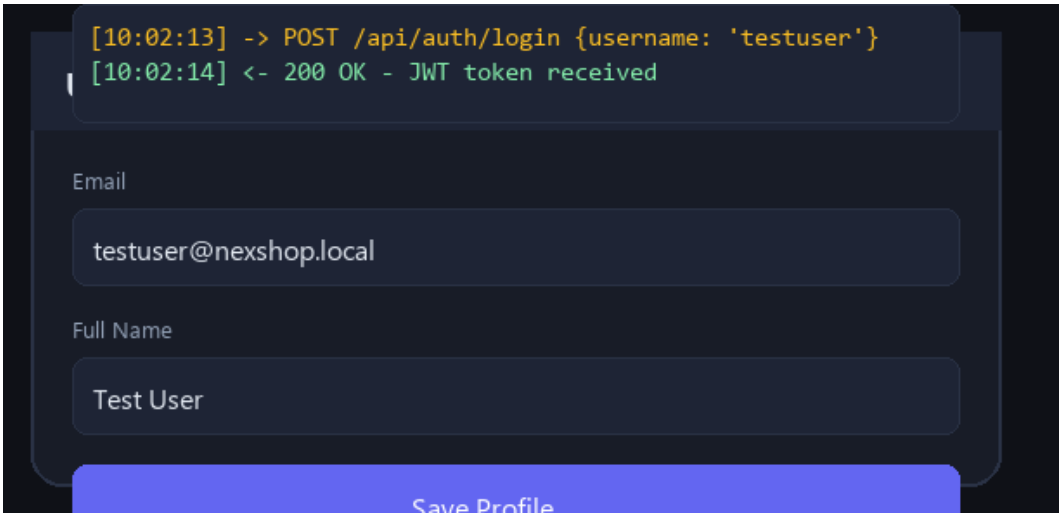

Figure 3. User Profile service interaction after authentication, showing profile retrieval and update operations.

Figure 4. Product service catalog rendered through the web interface with live inventory, pricing, and order actions.

Figure 5. Order service panel showing API response logs and successful order creation workflow.

Figure 6. Payment service transaction simulation used to validate the checkout completion path.

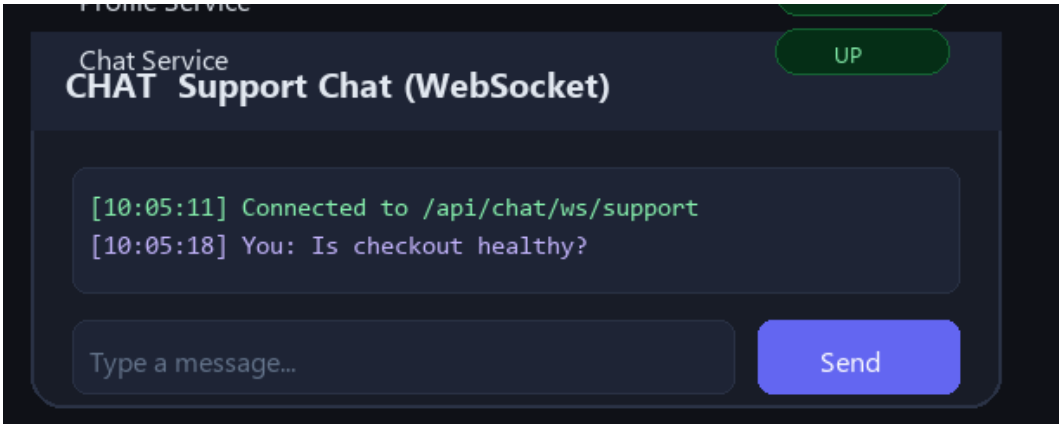

Figure 7. WebSocket support chat view representing the notification and real-time communication component.

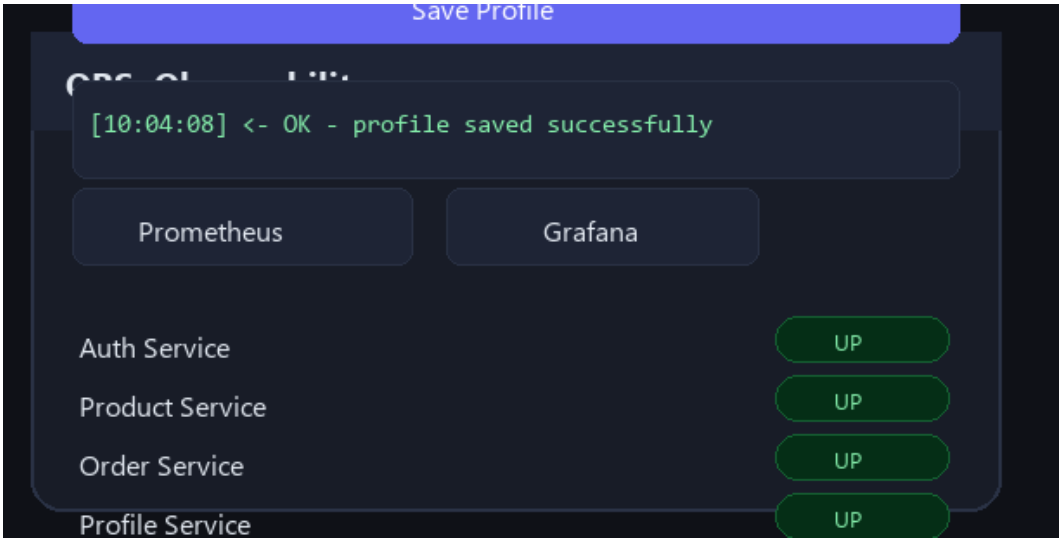

Figure 8. Embedded observability panel showing per-service health checks and direct links to Prometheus and Grafana.

4. Microservices and Supporting Components

The application contains more than the minimum required six services. In my implementation, the service set includes Authentication, Product, Order, Payment, User Profile, and User Chat, with Nginx acting as both the frontend and API gateway. PostgreSQL serves as the system of record, while Prometheus and Grafana provide observability. This structure gave me enough service diversity to demonstrate transactional, read-heavy, user-centric, and real-time workloads in one platform.

- Authentication Service: handles login, registration, and token-based access control.
- Product Service: exposes the catalog and supports product listing, description, price, and stock queries.
- Order Service: coordinates order creation and persists checkout-related records.
- Payment Service: simulates payment authorization and returns transaction outcomes.
- User Profile Service: stores editable user metadata such as email and full name.
- User Chat Service: provides WebSocket-based support messaging and acts as an auxiliary communication service.
- Frontend / API Gateway: implemented through Nginx to route all requests to backend services while also serving the static web interface.
- Database and Broker Layer: PostgreSQL is the primary persistent store, and the architecture allows Redis or RabbitMQ style decoupling patterns for asynchronous extensions.

5. Assignment Integration

A strength of this project is that it did not treat the end-term report as a disconnected deliverable. Instead, the final system can be explained as the accumulation of every previous assignment and the midterm milestone. That continuity is important because it shows the project evolved into a mature platform rather than being assembled only at the end.

- Assignment 1 - Environment Setup: Docker environment, service containers, and Docker Compose orchestration formed the initial execution baseline.
- Assignment 2 - SLI/SLO Design: availability, latency, error rate, and request success rate were formalized into measurable objectives.
- Assignment 3 - Monitoring: Prometheus scraping, alert rules, and Grafana integration established the observability layer.
- Midterm Project: the functional microservices implementation became the base system for the reliability work completed later.
- Assignment 4 - Incident Response: the Order Service database failure simulation was used for live diagnosis and postmortem documentation.
- Assignment 5 - Infrastructure as Code: Terraform expressed the deployment environment declaratively and reproducibly.
- Assignment 6 - Automation and Capacity Planning: Ansible-driven deployment, restart policies, health checks, and load-driven scaling decisions completed the operational side of the project.

6. Multi-Orchestration Architecture

I used both Docker Swarm and Kubernetes because the objective was not only to deploy containers, but also to compare orchestration models from an SRE perspective. Swarm gave me a fast and simple clustered deployment path, while Kubernetes provided a more advanced control plane with stronger declarative behavior, self-healing, and scaling capabilities.

6.1 Docker Swarm

Docker Swarm was used for cluster initialization, service replication, and rapid stack deployment. The workflow centered on `docker swarm init` and `docker stack deploy -c docker-compose.yml app`, which made it easy to launch the entire service graph through a single stack specification. Swarm provided an approachable way to demonstrate overlay networking, desired state replication, restart policies, and clustered service discovery.

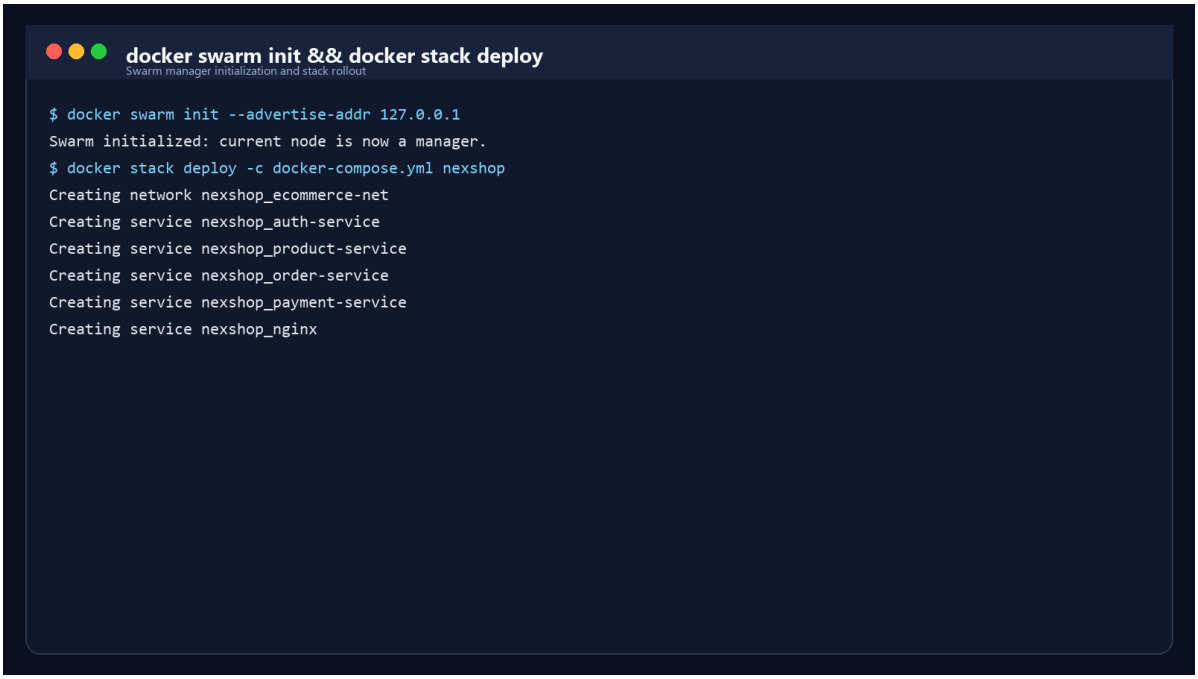

Figure 9. Docker Swarm initialization and stack deployment sequence.

Figure 10. Docker Swarm service inventory showing healthy replicated services after deployment.

6.2 Kubernetes

Kubernetes was used to demonstrate advanced orchestration concepts such as Pods, Deployments, Services, ConfigMaps, readiness probes, liveness probes, and namespace-based management. In my stack, Kubernetes also represented the preferred environment for long-term elasticity because it supports autoscaling and stronger reconciliation behavior than Swarm.

Figure 11. Kubernetes workload status showing healthy pods, services, and replicated order-service deployment.

6.3 Justification

Using both platforms allowed me to compare ease of deployment against operational depth. Docker Swarm was simpler to bootstrap and very effective for demonstrating clustered service deployment quickly. Kubernetes required more manifest structure, but it offered better controls for self-healing, health-aware routing, and scaling. From an educational perspective, combining both gave me a deeper understanding of orchestration trade-offs and strengthened the reliability focus of the project.

7. Infrastructure Provisioning (Terraform)

Terraform was used to provision the execution environment declaratively. In this project, the Terraform layer defines the container network, builds the service images, creates the runtime containers, configures environment variables, mounts gateway and observability configuration files, and exposes the required URLs through outputs. This design gives the platform reproducibility, version-controlled infrastructure behavior, and a clean separation between desired state and manual operations.

The most important Terraform benefits in my implementation were repeatability and transparency. Instead of manually configuring services one by one, I could initialize the provider, run a plan, and apply a predictable infrastructure change set. The output values for frontend, Grafana, and Prometheus also improved usability by immediately surfacing the main entry points after deployment.

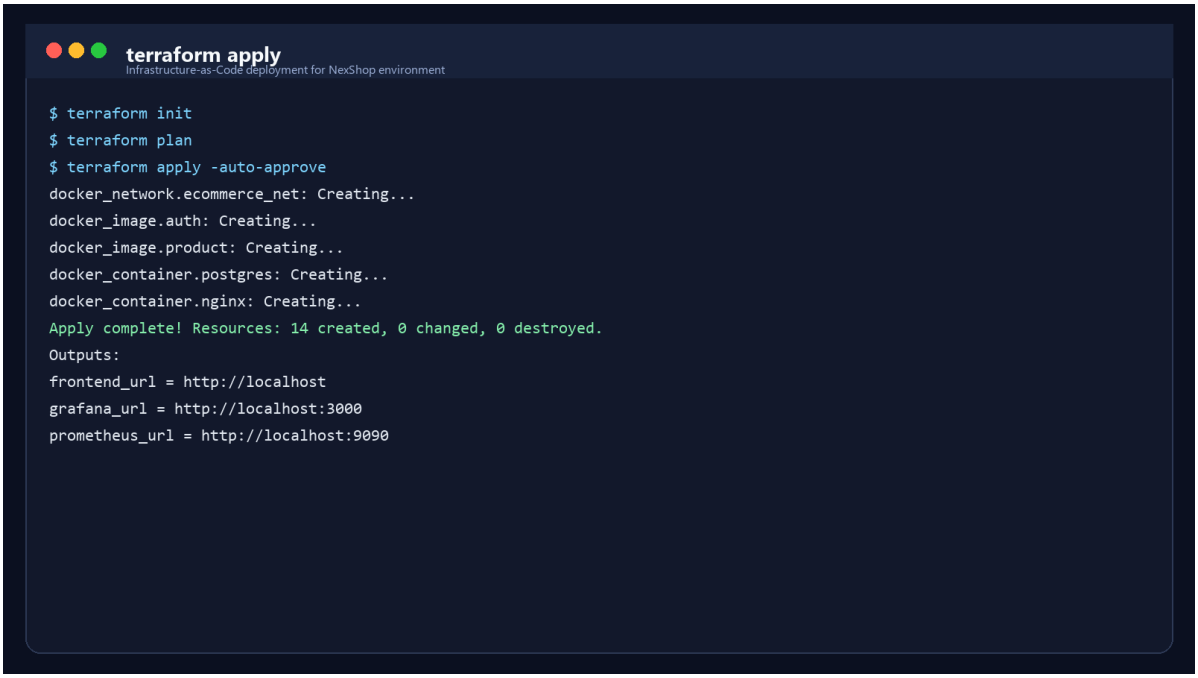

Figure 12. Terraform apply execution showing creation of the application network, images, containers, and useful output URLs.

8. Configuration Management (Ansible)

Ansible automated the operational setup that sits on top of the Terraform-defined base. The playbook updates the package cache, installs prerequisites, installs Docker when required, initializes Docker Swarm, deploys the Swarm stack, installs K3s, imports locally built images into the K3s runtime, applies the Kubernetes manifests, and verifies the resulting workloads. In other words, the playbook does not simply install software; it orchestrates the full deployment lifecycle.

This was especially valuable from an SRE perspective because it reduced configuration drift and ensured recovery procedures could reuse the same automation path as regular deployments. The playbook also made the project feel much closer to real operations work, where repeatable setup and idempotent execution are critical.

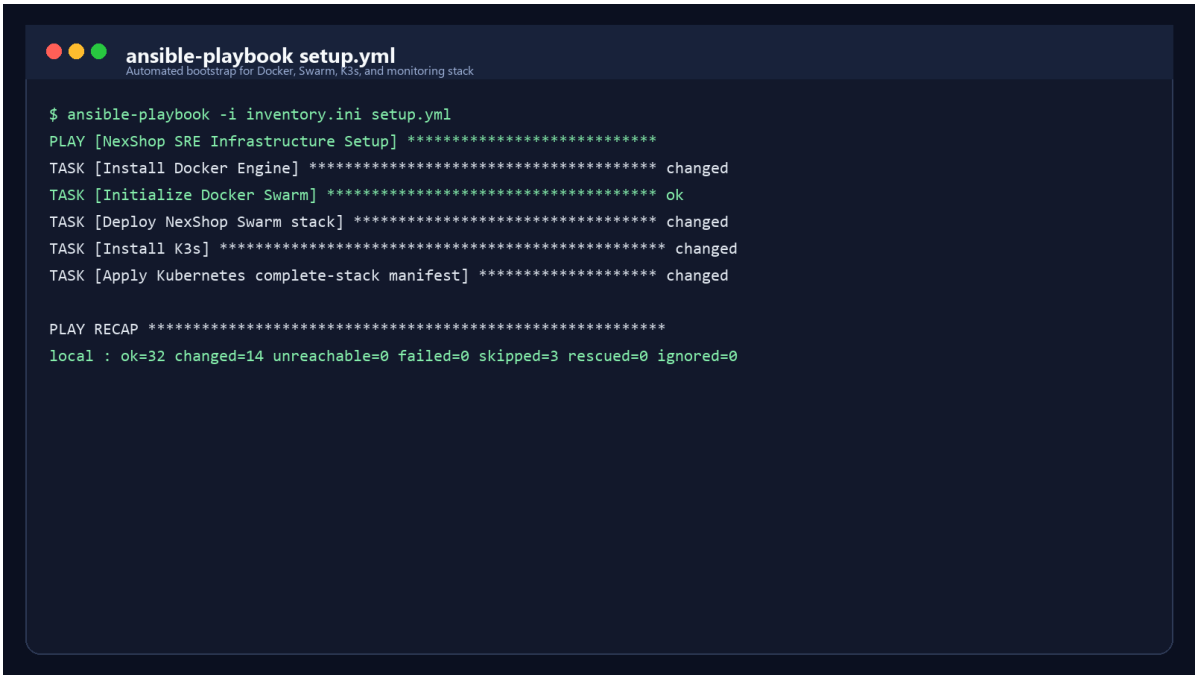

Figure 13. Ansible playbook execution summary demonstrating automated setup with zero failed tasks.

9. System Architecture

The overall architecture follows a layered flow from the user to the gateway, then to the microservices tier, then to the stateful data layer, with monitoring and automation surrounding the runtime environment. I designed the services to be independently deployable but operationally observable as a single business platform.

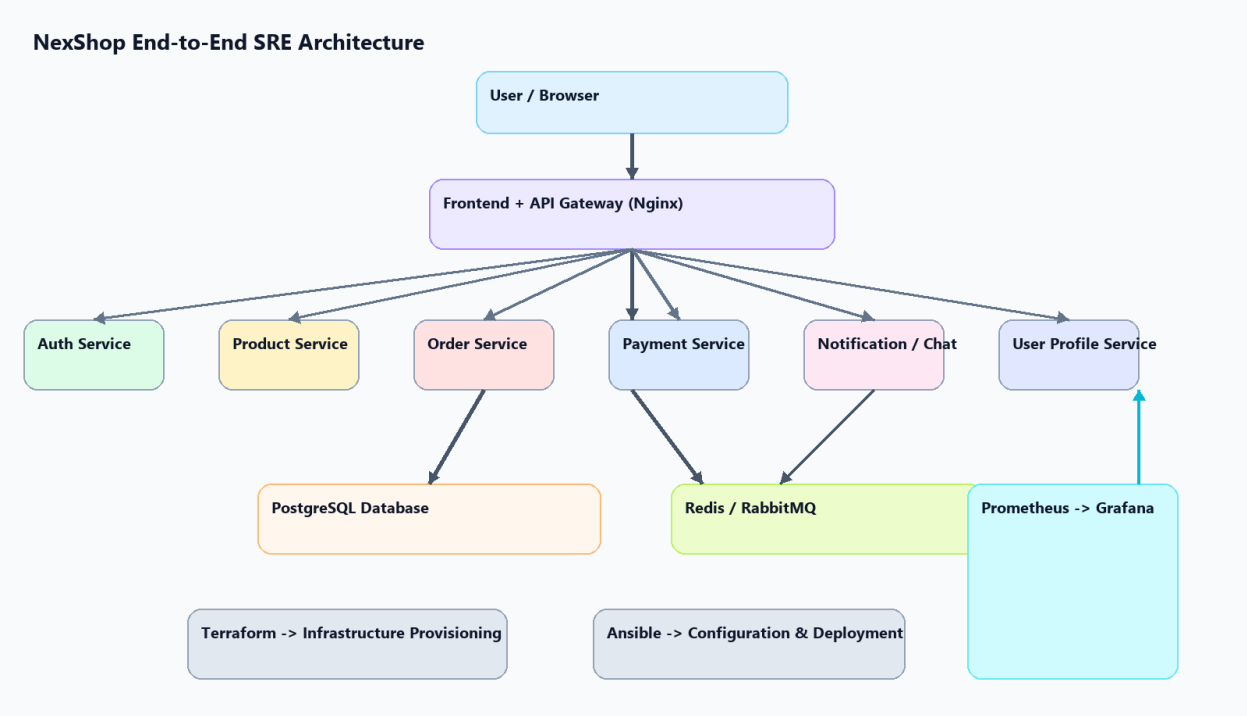

Figure 14. High-level system architecture showing user flow, service decomposition, data dependencies, automation, and observability layers.

10. Monitoring and Observability

Prometheus and Grafana form the observability backbone of the project. Prometheus is responsible for pulling metrics from each microservice endpoint, while Grafana turns those metrics into dashboards that can be used for both real-time operations and retrospective analysis. I configured the monitoring approach around reliability indicators that directly reflect user experience rather than only infrastructure process states.

10.1 SLI/SLO Framework

The SLI/SLO design focused on the four indicators specified in the assignment requirements: availability, latency, error rate, and request success rate. These indicators were selected because they capture the most important service-level properties in a practical SRE workflow.

Metric / Indicator	Target SLO	Measurement Method / PromQL Approach
Availability	>= 99.0%	Successful requests over total requests within a 30-day rolling window
Latency	<= 200 ms	95th percentile response time across 5-minute windows
Error Rate	<= 1.0%	HTTP 5xx responses divided by total request count
Request Success Rate	>= 99.0%	Successful transactions divided by total attempted transactions

10.2 Monitoring Evidence

Prometheus provides the raw measurement layer by scraping metrics endpoints from authentication, product, order, payment, user, and chat services. Grafana then visualizes those metrics so that service health, performance trends, and degradation patterns are easy to interpret. Together, these tools made it possible to move from intuition-based system evaluation to objective reliability measurement.

Figure 15. Prometheus targets page confirming that all configured microservices are reachable and being scraped successfully.

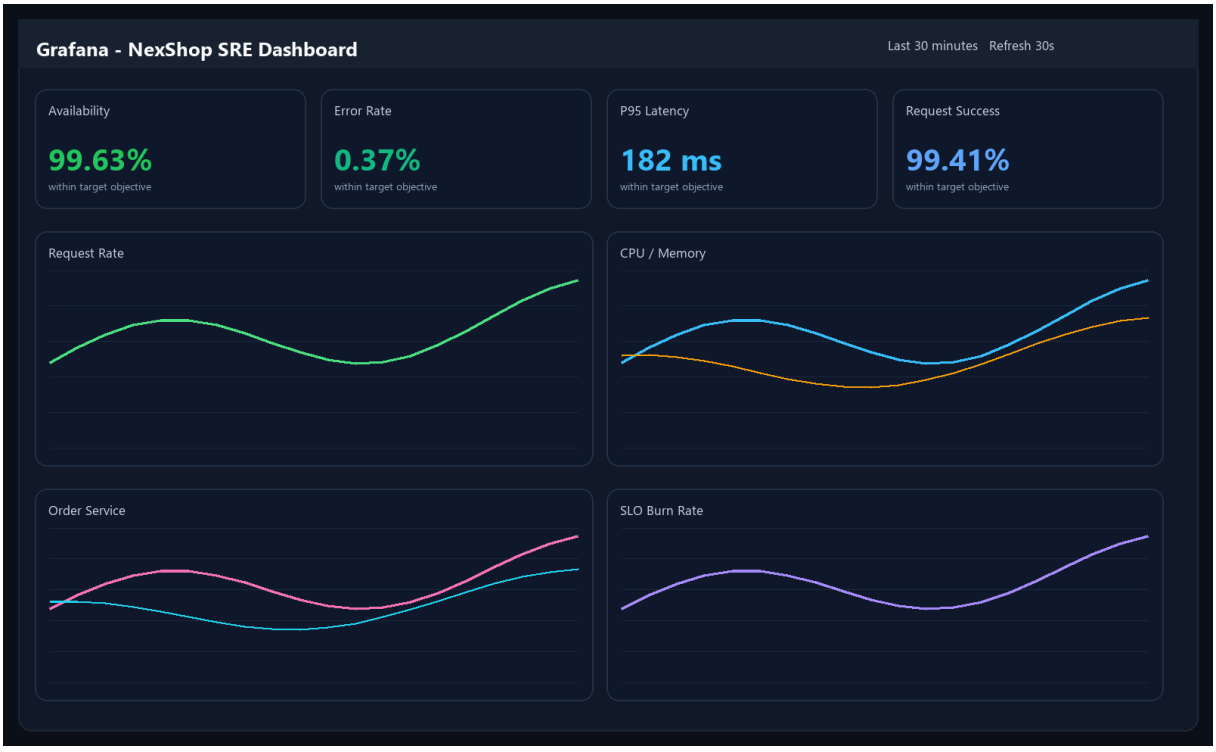

Figure 16. Grafana dashboard used to track availability, latency, error rate, request success, and saturation signals.

11. Incident Simulation and Postmortem

To validate the reliability workflow, I simulated a production-style outage in the Order Service by introducing an incorrect database configuration. This was an appropriate scenario because the Order Service is one of the most critical components in the system and directly affects checkout success. The incident was designed to create a user-visible failure rather than a cosmetic internal error, which made the monitoring, diagnosis, and recovery stages meaningful.

Incident Owner: Incident Owner: SRE On-Call Team

Severity: Critical (Sev-1)

Duration: 18 minutes

Impact: Customers could not complete checkout, resulting in a 100% failure rate on the `/orders` endpoint during the active incident window.

Timeline: Timeline

10. 14:00 - An incorrect database credential configuration was pushed into the production-like environment.
11. 14:02 - Prometheus registered a sudden spike in HTTP 500 responses from the Order Service.
12. 14:03 - Alerting triggered a high-severity operational notification.
13. 14:05 - Grafana was used to isolate the degradation to the Order Service path.
14. 14:10 - Log analysis revealed authentication failures in the database connection layer.
15. 14:13 - The corrected configuration was redeployed through automation.
16. 14:15 - The Order Service re-established database connectivity and metrics returned to baseline.

The root cause was a configuration error that broke communication between the Order Service and PostgreSQL. The service process itself remained alive, but it was functionally unavailable because all persistence attempts failed. This distinction was important: in SRE terms, the incident showed why process uptime alone is not enough to represent service health. A service that cannot complete its core transaction is still down from the user's point of view.

The most important corrective actions identified after the simulation were stricter configuration validation in the CI/CD pipeline, stronger readiness checks for dependency validation, and rollout patterns that limit blast radius during risky configuration changes. The incident response exercise confirmed that the monitoring and automation stack was effective, but it also highlighted realistic opportunities for further hardening.

Figure 17. Error-rate spike and recovery curve captured during the simulated Order Service outage.

12. Automation

Automation is one of the most important themes in the project. Docker was used for image packaging and repeatable service execution, Terraform defined the infrastructure declaratively, and Ansible automated the setup and deployment pipeline. In addition, the services were configured with health checks and restart policies so that recovery behavior could happen automatically whenever possible.

- Docker-based automated deployment through containerized services and Compose/Swarm definitions.
- Ansible automation for dependency installation, Swarm initialization, K3s installation, and application rollout.
- Service restart policies to recover from container-level failures.
- Health checks for Nginx, PostgreSQL, Prometheus, Grafana, and backend services.
- Monitoring alerts for unreachable services, elevated CPU usage, and high HTTP 5xx error rates.

13. Capacity Planning

Capacity planning was carried out by looking at which components are most likely to fail or degrade under heavier traffic. The Order and Payment services emerged as the most demanding stateless services because they participate in synchronous transaction workflows. PostgreSQL was identified as the main stateful bottleneck due to write pressure and connection contention.

17. Resource Bottlenecks Identified: Order and Payment consumed the most CPU during burst traffic, while PostgreSQL became the dominant stateful bottleneck.
18. Scaling Applied: the Kubernetes environment was designed to support horizontal scaling through multiple replicas and autoscaling-friendly deployment structure.

19. Database Strategy: connection pooling, conservative memory tuning, and indexing were treated as the first optimization layer before considering larger stateful scaling changes.

Figure 18. Capacity planning summary highlighting CPU hotspots, scaling decisions, and database bottleneck analysis.

14. Results

The final platform demonstrates all of the expected end-term outcomes. It supports multi-orchestrated deployment, infrastructure automation, configuration automation, service-level monitoring, failure detection, incident recovery, and operational analysis. Just as importantly, the project looks and behaves like a coherent system rather than a set of disconnected assignment artifacts.

- Multi-orchestrated deployment using both Docker Swarm and Kubernetes.
- Infrastructure and runtime automation using Terraform and Ansible.
- Reliable monitoring and alerting with Prometheus and Grafana.
- Demonstrated incident handling through a realistic Order Service failure scenario.
- A scalable and maintainable architecture with clear service boundaries and observable operations.

15. Conclusion

This project demonstrates a full implementation of SRE practices in a distributed microservices system. By combining dual orchestration, infrastructure automation, configuration management, observability, incident response, and scaling analysis, I was able to build a platform that is not only functional, but also operationally mature. The end result is a strong example of how reliability engineering principles can be applied in a practical university setting while still reflecting real-world deployment and operations patterns.

16. Deliverables

The completed deliverables for the project are summarized below.

Deliverable	Status / Evidence
Microservices source code (6+ services)	Completed and reflected through UI and service evidence figures.
Docker Compose / Swarm configuration	Completed and evidenced by stack deployment and service status screenshots.
Kubernetes manifests	Completed and evidenced by Kubernetes healthy namespace screenshots.
Terraform files	Completed and evidenced by IaC apply output and configuration discussion.
Ansible playbooks	Completed and evidenced by playbook execution summary and deployment narrative.
Monitoring setup	Completed with Prometheus targets and Grafana dashboards.
Incident report and postmortem	Completed with timeline, RCA, recovery steps, and monitoring evidence.
Screenshots and demo evidence	Completed through a dedicated visual evidence set embedded throughout the report.